Nosh CV Intern Task

Maulik, 31 October 2026
Code: github.com/maulik0jain/NoshInternTask
Tracking videos: https://drive.google.com/drive/folders/14QTbN2sMEghD2fOjj1JwozyTFmwKRztc
The full report under report/ in the repo has every trial with its parameters and all the plots.

Ground truth

- The dataset has 110 photos at 640x640 with class labels only, so there were no masks to compare against. SAM 2.1 needs a prompt, so I used Grounding DINO with the text "egg." to get boxes and passed each box to SAM.
- My first prompt was "an egg." and it missed one egg in each of two images, because those eggs scored 0.28 and 0.29 against a 0.30 threshold. I only noticed because I went through all 110 overlays. Switching to "egg." fixed both.
- In two other images the detector took an egg's reflection in a glossy lid for a second egg. I removed those by hand in an override file and wrote down why, since a rule made for two images would only fit those two. One image (#17) still misses a piece of egg behind a stem, which I left as a known error.

Splitting the data

- A random split looked wrong once I went through the photos. They were taken in sessions, so the same pot or keyboard turns up again from a slightly different angle, and a random split would have me tuning and testing on the same scene.
- I grouped the photos by scene with DINOv2 embeddings (35 groups), then made 8 merges by hand where one physical setup had been split across groups. That left 19 scenes: 85 dev images from 14 scenes and 25 eval images from 5.
- My first two splits were bad. One had only 4 scenes in eval, and the next had a single multi-egg image in eval, which would have made eval too easy.
- As a check, a random split of the same size gave 16% of eval images a near twin in dev. Mine gave 0%.

Round 1: the classical pipeline

- Otsu on grayscale was the baseline at a dev IoU of 0.366. It found the eggs along with every other bright thing in the photo. Gray-world white balance and CLAHE made it worse (0.266), since they brighten the background as much as the egg.
- Shape filters after a single threshold were the most frustrating failure. IoU fell to 0.312 and 36 images came out empty, because at that one threshold the egg was usually stuck to some bright background and the filter threw both away.
- After that I stopped relying on one threshold. The search tries 24 thresholds per image and keeps the blob that looks most like an egg (ellipse fit times solidity). That gave 0.448, and 0.464 on the Lab L* channel, which had lost earlier with plain Otsu.
- GrabCut on top was the biggest single jump, to 0.566, because the search only found the lit top of each egg and GrabCut grew it out to the shaded edge. Snapping masks to a perfect ellipse made edges worse, since a real egg has one blunt end. A fallback pass for empty images brought the round 1 pipeline (T11) to 0.589.
- I also hit real bugs. GrabCut gave a different answer on every run until I fixed its random seed, and one bad ellipse fit tried to allocate 71 GB.
- Eval, run once, gave an IoU of 0.618 (median 0.817). The method does well when the egg is the brightest egg-shaped thing in the picture and has almost no colour, and fails completely otherwise. White bags, fridge trays and pale walls were the worst, and all three eval photos of a backlit hand in a dark room scored 0.
- An ablation afterwards showed the skin filter I had added early on was now hurting (dev 0.609 without it). I left it in, because changing it after seeing eval would have spoiled the eval number.

Round 2: a dev gain that didn't reach eval

- I took the five fixes from my failure analysis and tried each one on dev. Dev IoU went from 0.589 to 0.669, and I was happy with that until eval came back at 0.613, down from 0.618.
- Only 1 of the 11 eval failures got better. Every change had been kept or dropped on dev, and none of the eval scenes were in dev, so the fixes didn't carry over.
- One change, scoring blobs by edge support, also had a plain design mistake. It pushed every score below the cut, and 33 images came out empty before I fixed it.

Round 3: trying to stop the overfitting

- I suspected two causes: around 40 decisions made on the same 85 images, and big scenes counting for more than small ones.
- So I changed how I decided. Each change was judged per scene, with leave-one-scene-out to estimate how the selection would do on a scene it had never seen, and I committed the rule before running anything.
- Only 3 of the 5 round 2 changes passed. The round 3 pipeline (T21) scored 0.655 on dev and 0.612 on eval, about the same as before. Its held-out scene estimate was 0.615, within 0.003 of eval, while round 2's dev number had been 0.056 too high.
- The stricter rule was less stable than I expected. With only 14 scenes, borderline changes kept flipping in and out between folds.

The video: round 3 did worse than round 1

- For Task 4 I ran T21 with a simple tracker that matches masks frame to frame with the Hungarian method and keeps a lost egg's id for 2 seconds. Against SAM 2.1 video tracking it got MOTA 0.159 and IDF1 0.216, which was disappointing.
- The tracker only lost 12 eggs that segmentation had found, so most of the error was in segmentation. The table is glossy and each egg sits on a white, egg-shaped reflection, so the two touching eggs merged with their reflections into one blob. Eggs under the hand weren't found at all.
- None of my rounds had seen this video, so I used it as one more fresh test. Round 1 scored 0.868 IoU, round 2 scored 0.677 and round 3 scored 0.681, so the changes I had kept on dev made things clearly worse here.
- Turning the skin filter off caused most of the drop: that step alone cost 0.130 of the 0.187 gap. In this video the skin colour box covers 95% of the table under the eggs, so with the filter on, the reflections are removed before the egg search sees them. On the photos the same box was deleting warm-lit eggs, which is why I had turned it off.
- Round 1 with the same tracker reached MOTA 0.826 and IDF1 0.605. I ran that after seeing the fresh test, so it isn't a clean number and I report it separately.

What I'd take away

- A gain on dev told me little about eval, because the eval scenes were never in dev, and 14 scenes were too few to trust a keep or drop call on a colour filter that helps one kind of scene and breaks another. Next time I'd judge every change on held-out scenes from the start.
- I would also make the skin filter depend on the scene, reject blobs that sit right under a brighter egg (reflections), and carry masks forward with optical flow while an egg is under the hand.

Where I used AI

- The decisions in this project were mine, from what to try to what to keep. Claude Code did the building: at each step I told it what I wanted, ran what it wrote, looked at the outputs and decided what came next.
- Before starting, I used Claude in chat to understand the task, read up on classical segmentation and help write the step-by-step plan I gave Claude Code.

What I worked out and decided

- That a random split would leak scenes, which photos belonged together, and the 8 merges I made by hand
- Going through all 110 ground truth overlays, which is how I found the prompt problem and the lid reflections, and choosing a written override over a rule fitted to two images
- What to try in each trial and whether to keep it, after looking at the worst-case images from every run
- The bugs, which I found by looking at the outputs
- The cause of each failed image, seeing that dev gains weren't reaching eval, and the round 3 idea of judging per scene with leave-one-scene-out
- Using the video as a fresh test and tracing the drop back to the skin filter and the reflections

What Claude built

- The ground truth script with Grounding DINO and SAM 2.1, and the overlay sheets
- The scene grouping, the grouped split and the leakage check
- Every stage of the classical pipeline, plus the experiment runner, sweeps and metrics
- The analysis, ablation and leave-one-scene-out scripts and the plots
- The SAM 2.1 video reference, the tracker and the video export
- Drafts of the report text, written from my results. Every number in it comes from result files the scripts saved.